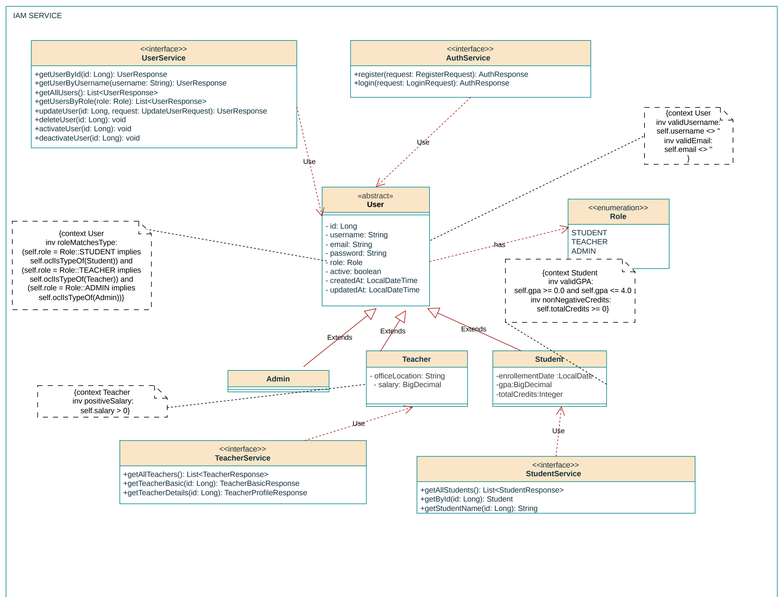
IAM SERVICE
<<interface>>
UserService
+getUserById(id: Long): UserResponse
+getUserByUsername(username: String): UserResponse
+getAllUsers(): List<UserResponse>
+getUsersByRole(role: Role): List<UserResponse>
+updateUser(id: Long, request: UpdateUserRequest): UserResponse
+deleteUser(id: Long): void
+activateUser(id: Long): void
+deactivateUser(id: Long): void
<<interface>>
AuthService
+register(request: RegisterRequest): AuthResponse
+login(request: LoginRequest): AuthResponse
{context User
inv validUsername:
self.username <> ''
inv validEmail:
self.email <> ''
}
Use
Use
«abstract»
User
- id: Long
- username: String
- email: String
- password: String
- role: Role
- active: boolean
- createdAt: LocalDateTime
- updatedAt: LocalDateTime
has
<<enumeration>>
Role
STUDENT
TEACHER
ADMIN
{context User
inv roleMatchesType:
(self.role = Role::STUDENT implies
self.oclIsTypeOf(Student)) and
(self.role = Role::TEACHER implies
self.oclIsTypeOf(Teacher)) and
(self.role = Role::ADMIN implies
self.oclIsTypeOf(Admin))}
{context Student
inv validGPA:
self.gpa >= 0.0 and self.gpa <= 4.0
inv nonNegativeCredits:
self.totalCredits >= 0}
Extends
Extends
Extends
Admin
Teacher
- officeLocation: String
- salary: BigDecimal
Student
-enrollementDate :LocalDate
-gpa:BigDecimal
-totalCredits:Integer
{context Teacher
inv positiveSalary:
self.salary > 0}
Use
Use
<<interface>>
TeacherService
+getAllTeachers(): List<TeacherResponse>
+getTeacherBasic(id: Long): TeacherBasicResponse
+getTeacherDetails(id: Long): TeacherProfileResponse
<<interface>>
StudentService
+getAllStudents(): List<StudentResponse>
+getById(id: Long): Student
+getStudentName(id: Long): String
Academic-coreService
«microservice»
IAM Service
«ref: userId»
«ref: studentId»
«ref: teacherId»
{context Feedback
inv hasUser:
self.userId <> null
inv hasCourse:
self.courseId <> null
inv nonEmptyComment:
self.comment <> ''}
{context Enrollment
inv noDuplicateEnrollment:
Enrollment.allInstances()->forAll(e1, e2 |
e1 <> e2 implies
not (e1.studentId = e2.studentId and
e1.courseId = e2.courseId))}
Feedback
- id: Long
- userId: Long
- courseId: Long
- comment: String
- createdAt: LocalDateTime
Enrollment
- id: Long
- studentId: Long
- courseId: Long
- enrolledAt: LocalDateTime
+ create(studentId: Long, courseId: Long): Enrollment
«publishes»
«event»
StudentEnrolled
- studentId: String
- enrolledCourseId: String
- courseName: String
{context Course
inv validDateRange:
self.startDate.isBefore(self.endDate)
inv capacityNotExceeded:
self.enrolledCount <= self.maxStudents
inv positiveCredits:
self.credits > 0
}
{context Course::isFull()
post: result = (self.enrolledCount = self.maxStudents)}
Course
- id: Long
- name: String
- courseCode: String
- description: String
- startDate: LocalDate
- endDate: LocalDate
- createdAt: LocalDate
- credits: int
- maxStudents: int
- enrolledCount: int
- departmentId: Long
- teacherId: Long
+ Course()
+ isFull(): boolean
+ enrollStudent(): void
+ unenrollStudent(): void
belongs to
Department
- id: Long
- name: DepartmentsType
Use
<<enum>>
DepartmentsType
Computer_Science
Information_Systems
Software_Engineering
Artificial_Intelligence
Data_Science
Cybersecurity
Information_Technology
{context Course::enrollStudent()
pre courseNotFull:
self.enrolledCount < self.maxStudents
post countIncremented:
self.enrolledCount = self.enrolledCount@pre + 1}
{context Course::unenrollStudent()
pre hasStudents:
self.enrolledCount > 0
post countDecremented:
self.enrolledCount = self.enrolledCount@pre - 1}
Announcement
- id: Long
- title: String
- content: String
- courseId: Long
- createdAt: LocalDateTime
«publishes»
context Announcement
invariant announcementHasCourse:
self.courseId <> null
«event»
CourseCreatedEvent
- courseId: String
- courseName: String
- departmentName: String
- createdAt: LocalDate
«event»
AnnouncementCreatedEvent
- id: String
- courseId: String
- courseName: String
- title: String
- description: String
- createdAt: LocalDateTime
«publishes»
Communication Service
Message
- id: Long
- content: String
- createdAt: LocalDateTime
- updatedAt: LocalDateTime
- course: Course
- sender: User
{context Message
inv nonEmptyContent:
self.content <> ''
inv hasSender:
self.sender <> null
inv hasCourse:
self.course <> null}
Use
Read-only stub.
Real entity lives in
Academic Core
Service
«stub»
Course
- id: Long
- name: String
<<interface>>
MessageService
+ createMessage(messageRequest: MessageRequest): void
+ deleteMessageById(messageId: Long): void
+ getMessagesByCourseId(courseId: Long): List<MessageResponse>
+ getMessagesBySenderId(senderId: Long): List<MessageResponse>
+ countMessagesByCourseId(courseId: Long): long
«stub»
EnrolledCourse
- id: Long
- student: User
- course: Course
«stub»
User
- id: Long
- userName: String
-email: String
<<interface>>
NotificationService
+ createNotification(notificationRequest: NotificationRequest): NotificationResponse
+ markAsRead(notificationId: Long): NotificationResponse
+ markAllAsReadForUser(userId: Long): int
+ deleteNotificationById(notificationId: Long): void
+ deleteAllNotificationsForUser(userId: Long): void
+ sendNotificationToUser(notificationRequest: NotificationRequest): void
+ sendNotificationToCourse(notificationRequest: NotificationCourseRequest): void
+ getNotificationById(notificationId: Long): NotificationResponse
+ getAllNotificationsForUser(userId: Long): List<NotificationResponse>
+ getUnreadNotificationsForUser(userId: Long): List<NotificationResponse>
+ getNotificationsByType(userId: Long, type: NotificationType): List<NotificationResponse>
+ countUnreadForUser(userId: Long): long
Use
<<enum>>
NotificationType
ENROLLMENT
ANNOUNCEMENT
COURSE_UPDATE
GRADE
SYSTEM
Use
Notification
-id:Long
-recipient :User
-title :String
-message:String
-type:NotificationType
- isRead: boolean
- createdAt: LocalDateTime
- updatedAt: LocalDateTime
{context Notification
invariant hasRecipient:
self.recipient <> null
invariant validType:
self.type <> null}